SQL Y PYTHON

ADMINISTRADOR
DE MÚSICA



ESFM – IPN

MODELADO DE BASE(ENTIDADES)

Para realizar este proyecto diseñamos la base de datos music_manager, dividiendo su estructura en tablas maestras y tablas intermedias. En primer lugar, creamos las siguientes entidades principales:

- **usuarios**: Almacena las credenciales de acceso (username y password_hash) y la fecha de registro para gestionar el inicio de sesión en el sistema.
- **artistas**: Registra el catálogo de músicos o bandas mediante un identificador único y su nombre.
- **albumes**: Guarda la información general de cada producción discográfica, incluyendo su título, descripción, año de lanzamiento y la ruta de su portada.
- **canciones**: Contiene los datos de cada pista musical, registrando su título, su duración en segundos y la ruta del archivo local.
- **playlists**: Permite crear listas de reproducción personalizadas almacenando su nombre, descripción y fecha de creación.
- **notas**: Funciona como un registro de apuntes o reseñas de texto con marca de tiempo para documentar comentarios dentro de la biblioteca.

Tabla albumes	
Dato	**Tipo de dato**
id_album	INT(150)
titulo	VARCHAR(150)
descripcion	TEXT
anio_lanzamiento	YEAR
portada_ruta	VARCHAR(225)

Tabla artistas	
Dato	**Tipo de dato**
id_artista	VARCHAR(150)

Tabla autores	
Dato	**Tipo de dato**
id_autor	INT
nombre	TEXT
nacionalidad	TEXT

TABLAS ENTIDADES

Tabla canciones	
Dato	**Tipo de dato**
id_cancion	VARCHAR(150)
titulo	VARCHAR(150)
duracipn_degundos	INT
ruta_archivo	VARCHAR(255)

Tabla notas	
Dato	**Tipo de dato**
id_nota	INT
contenido	INT
fecha	TIMESTAMP

Tabla usuario	
Dato	**Tipo de dato**
id_usuario	INT
username	VARCHAR(50)
password_hash	VARCHAR/(255)
created_at	TIMESTAMP

Tabla playlist	
Dato	**Tipo de dato**
id_playlis	INT
nombre	VARCHAR(100)
descripción	TEXT
created_at	TIMESTAMP

CODIGO EN SQL(ENTIDADES)

Código para usuarios:

```sql
CREATE TABLE usuarios (
    id_usuario INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

Código para artistas:

```sql
CREATE TABLE artistas (
    id_artista INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(150) NOT NULL
) ENGINE=InnoDB;
```

Código para albumes:

```sql
CREATE TABLE albumes (
    id_album INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    descripcion TEXT NULL,
    anio_lanzamiento YEAR NULL,
    portada_ruta VARCHAR(255) NULL
) ENGINE=InnoDB;
```

Código para canciones:

```sql
CREATE TABLE canciones (
    id_cancion INT AUTO_INCREMENT PRIMARY KEY,
    titulo VARCHAR(150) NOT NULL,
    duracion_segundos INT NOT NULL DEFAULT 0,
    ruta_archivo VARCHAR(255) NULL
) ENGINE=InnoDB;
```

Código para playlists:

```sql
CREATE TABLE playlists (
    id_playlist INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    descripcion TEXT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAM
) ENGINE=InnoDB;
```

Código para notas:

```sql
CREATE TABLE notas (
    id_nota INT AUTO_INCREMENT PRIMARY KEY,
    contenido TEXT NOT NULL,
    fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;
```

MODELADO DE BASE(RELACIONES)

Relación: Álbum - Canciones

```
CREATE TABLE IF NOT EXISTS rel_tracks_album (
    id_album INT NOT NULL,
    id_cancion INT NOT NULL,
    numero_pista INT NULL,
    PRIMARY KEY (id_album, id_cancion)
) ENGINE=InnoDB;
```

Relación: Playlist - Canciones

```
CREATE TABLE IF NOT EXISTS rel_tracks_playlist (
    id_playlist INT NOT NULL,
    id_cancion INT NOT NULL,
    posicion INT NULL,
    agregado_en TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (id_playlist, id_cancion)
) ENGINE=InnoDB;
```

Relación: Artista - Canción

```
CREATE TABLE IF NOT EXISTS rel_art_track (
    id_artista INT NOT NULL,
    id_cancion INT NOT NULL,
    rol VARCHAR(50) DEFAULT 'Principal',
)   PRIMARY KEY (id_artista, id_cancion)
```

Relación: Artista - Álbum

```
CREATE TABLE IF NOT EXISTS rel_art_album (
    id_artista INT NOT NULL,
    id_album INT NOT NULL,
    PRIMARY KEY (id_artista, id_album)
) ENGINE=InnoDB;
```

Relación: Notas - Álbum

```
CREATE TABLE IF NOT EXISTS rel_notes_album (
    id_nota INT NOT NULL,
    id_album INT NOT NULL,
    PRIMARY KEY (id_nota, id_album)
) ENGINE=InnoDB;
```

Relación: Usuario - Playlist

```
CREATE TABLE IF NOT EXISTS rel_user_playlist (
    id_usuario INT NOT NULL,
    id_playlist INT NOT NULL,
    es_propietario BOOLEAN DEFAULT TRUE,
)   PRIMARY KEY (id_usuario, id_playlist)
```

¿CÓMO FUNCIONAN LAS TABLAS RELACIONADAS?

En nuestro proyecto, las tablas de relación permiten conectar nuestros elementos entre sí de forma flexible sin repetir datos:

- **Relación Álbum - Canciones (rel_tracks_album):** Nos ayuda a que un álbum pueda tener varias canciones, y que una misma canción pueda pertenecer a varios álbumes (por ejemplo, en un álbum de estudio y en uno recopilatorio), asignándole su número de pista en cada uno.
- **Relación Playlist - Canciones (rel_tracks_playlist):** Permite que una playlist contenga muchas canciones, y que una misma canción pueda agregarse a diferentes playlists al mismo tiempo, guardando su posición y la fecha en que se añadió.
- **Relación Artista - Canción (rel_art_track):** Nos sirve para que un artista tenga múltiples canciones, y que una sola canción pueda tener varios artistas registrados (permitiendo colaboraciones o feats), especificando el rol de cada uno.
- **Relación Artista - Álbum (rel_art_album):** Facilita que un artista pueda lanzar varios álbumes a lo largo de su carrera, y que un álbum pueda pertenecer a más de un artista en caso de producciones conjuntas.
- **Relación Notas - Álbum (rel_notes_album)**: Permite que un álbum tenga varias notas o reseñas asociadas, vinculando cada comentario directamente con el álbum correspondiente.
- **Relación Usuario - Playlist (rel_user_playlist)**: Hace posible que un usuario cree o siga varias playlists, y que una misma playlist pueda estar vinculada a distintos usuarios, identificando quién es el propietario.

MODIFICACIONES PARA LAS TABLAS RELACIONALES

En el primer **CODIGO** definimos la estructura de todas nuestras tablas; sin embargo, hasta ese punto aún eran independientes entre sí. Para vincularlas formalmente, separamos la creación de las relaciones en el archivo MODIFICACIONES_BASE.sql, donde aplicamos los siguientes cambios:

LAS FUNCIONES DE SQL QUE USAMOS FUERON:

ALTER TABLE y ADD CONSTRAINT: Modificamos la estructura de las 6 tablas intermedias sin necesidad de borrarlas ni volverlas a crear.

Asignación de Llaves Foráneas (FOREIGN KEY ... REFERENCES):
Conectamos los identificadores (id_) de cada tabla intermedia directamente con la llave primaria de su tabla maestra correspondiente.

Reglas en cascada (ON DELETE CASCADE y ON UPDATE CASCADE):
Configuramos la base de datos para que, si se elimina o actualiza un registro principal (como un álbum, canción o usuario), sus vínculos se actualicen o eliminen automáticamente en las tablas de relación.
Versión

¿CÓMO RELACIONAMOS LAS TABLAS?

- En **rel_tracks_album**: Conectamos id_album con la tabla albumes e id_cancion con la tabla canciones.
  - Ejemplo: Así vinculamos la canción "No Surprises" o "Karma Police" con el álbum "OK Computer" asignándole su número de pista correspondiente, sin tener que volver a escribir los datos del disco en cada canción.
- En **rel_tracks_playlist**: Vinculamos id_playlist con playlists e id_cancion con canciones.
  - Ejemplo: Permite agregar canciones como "El Triste" o "La Nave del Olvido" a una playlist llamada "Clásicos en Español", guardando en qué posición de la lista se reproducen.
- En **rel_art_track**: Enlazamos id_artista con artistas e id_cancion con canciones.
  - Ejemplo: Relaciona a DLD con la canción "Todo Cuenta" indicando su rol como artista 'Principal', o permite registrar a dos artistas distintos cuando hacen una colaboración (feat.) en una misma pista.
- En **rel_art_album**: Unimos id_artista con artistas e id_album con albumes.
  - Ejemplo: Conecta directamente a Radiohead con sus álbumes "In Rainbows" y "The Bends", o a Luis Miguel con el álbum "Romances", estableciendo la autoría de cada obra.
- En **rel_notes_album**: Relacionamos id_nota con notas e id_album con albumes.
  - Ejemplo: Permite asociar una nota o reseña concreta como "Excelente producción y arreglos de cuerdas, mi disco favorito de 1997" directamente al álbum "OK Computer".
- En rel_user_playlist: Conectamos id_usuario con usuarios e id_playlist con playlists.
  - Ejemplo: Vincula el perfil de un usuario registrado (por ejemplo, el usuario joshua) con su playlist "Para estudiar y programar", marcando el campo es_propietario como True para indicar que él la creó.

rel_art_album	
Dato	**Tipo de dato**
id_artista	INT
id:album	INT

rel_art_track	
Dato	**Tipo de dato**
id_artista	INT
id_album	INT
rol	VARCHAR(50)

rel_notes_album	
Dato	**Tipo de dato**
id_nota	INT
id_album	INT

rel_art_track	
Dato	**Tipo de dato**
id_album	INT
id_canción	INT
número_pista	INT

rel_user_playlist	
Dato	**Tipo de dato**
id_ususario	INT
id_playlist	INT
es_propietario	TINYINT(1)

rel_art_track	
Dato	**Tipo de dato**
id_playlis	INT
id_canción	INT
posición	INT
agregado_en	TIMESTAMP

ARCHIVOS PYTHON

Para integrar el proyecto utilizamos Python, lo que nos permitió crear una interfaz en consola mucho más intuitiva y gestionar toda la base de datos sin que el usuario tenga que escribir comandos SQL directamente. Para mantener el código organizado, dividimos el sistema en 3 archivos .py.

- **main.py:** Es la interfaz principal e interactiva del programa. Primero muestra una pantalla de bloqueo para iniciar sesión o registrar un usuario nuevo; una vez verificadas las credenciales, despliega el menú principal con 9 opciones para agregar datos, vincular canciones a álbumes o consultar el catálogo en pantalla.

- **funciones.py:** Es el motor lógico que traduce las acciones del usuario en consultas SQL (INSERT y SELECT). Contiene las funciones para registrar usuarios, artistas, álbumes, canciones y playlists, enlazar tablas intermedias y mostrar la información formateada (como convertir la duración de segundos a minutos o unir álbumes con sus pistas mediante LEFT JOIN).

- **conexion.py:** Es el puente de comunicación con el servidor local de MySQL. Utiliza la librería mysql.connector para autenticar las credenciales de acceso y conectarse a la base de datos music_manager en el puerto 3306, manejando posibles errores de conexión mediante bloques try-except

CONEXION CON BASE DE DATOS (CONEXION.PY)

```
importar MySQL.conector
de MySQL.conector importar Error

definición conectar_a_la_base_de_datos():
    intentar:
        # 1. INTENTO DE CONEXIÓN
        cnx = MySQL.conector.conectar(
            anfitrión="localhost",
            usuario="raíz",
            contraseña="joshwbe",# Tu contraseña real
            base de datos="gestor_musical",
            puerto=3306
        )

        si cnx.está_conectado():
            imprimir("¡Conexión exitosa a la base de datos music_manager!")
            devolver cnx  # Esta es la instrucción clave para entregar la conexión viva

    excepto Error como error:
        # 2. CAPTURAMOS EL ERROR SI OCURRE
        imprimir(f"Error al conectar con MySQL:{error}")
        devolver Ninguno
```

¿Qué intentamos hacer? Establecer un canal de comunicación directo entre nuestro programa en Python y el servidor local de MySQL (localhost en el puerto 3306) para poder leer y guardar cambios dentro de la base de datos music_manager.
¿Qué es mysql.connector? Es el módulo o librería oficial que actúa como "traductor" entre Python y MySQL. Nos permite enviar nuestras credenciales de acceso (user, password, database) y verificar con is_connected() si la base de datos está activa y lista para recibir consultas.
¿Qué significa el bloque try - except? Es una estructura de control de errores. Con try (intentar) le pedimos a Python que intente conectarse al servidor; si ocurre algún fallo (por ejemplo, si el servidor de MySQL está apagado o la contraseña es incorrecta), el bloque except (excepción) captura el error y muestra un mensaje en pantalla en lugar de que el programa colapse abruptamente.

CONEXION CON BASE DE DATOS (FUNCIONES.PY)

Este archivo contiene las 11 funciones encargadas de registrar, vincular y mostrar la información de nuestra biblioteca musical:

Funciones de Usuarios:
add_user: Registra un nuevo usuario en la base de datos guardando su nombre de usuario y contraseña.
verify_user: Comprueba si el usuario y la contraseña ingresados existen y coinciden para permitirle iniciar sesión.
Funciones para Agregar y Vincular Datos:
add_artist: Guarda un nuevo artista o banda en el catálogo.
add_album: Registra un álbum nuevo con su título, descripción, año de lanzamiento y portada, devolviendo el ID asignado.
add_track: Agrega una nueva canción guardando su título, duración en segundos y ruta del archivo.
link_track_to_album: Conecta el ID de una canción con el ID de un álbum en la tabla intermedia y le asigna su número de pista.
add_playlist: Crea una nueva lista de reproducción y automáticamente la vincula con el usuario activo marcándolo como su propietario.

Funciones para Mostrar el Catálogo:
show_artists: Imprime en pantalla la lista de todos los artistas registrados con su ID.
show_albums: Muestra el catálogo completo de álbumes junto con las canciones que tiene vinculadas cada uno, ordenadas por número de pista.
show_tracks: Enlista todas las canciones registradas convirtiendo su duración de segundos a formato de minutos:segundos.
show_playlists: Muestra todas las listas de reproducción creadas junto con su ID y descripción.

CODIGO

```python
from conexion import connect_to_database

# ========================================
# FUNCIONES DE USUARIOS
# ========================================
def add_user(username, password_hash):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "INSERT INTO usuarios (username, password_hash) VALUES (%s, %s)"
            cursor.execute(sql, (username, password_hash))
            cnx.commit()
            print(f"Usuario '{username}' agregado exitosamente.")
        except Exception as e:
            print(f"Error al agregar el usuario: {e}")
        finally:
            cursor.close()
            cnx.close()

def verify_user(username, password_hash):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "SELECT id_usuario FROM usuarios WHERE username = %s AND password_hash = %s"
            cursor.execute(sql, (username, password_hash))
            resultado = cursor.fetchone()

            if resultado:
                return resultado[0]
            return None
        except Exception as e:
            print(f"Error al verificar usuario: {e}")
            return None
        finally:
            cursor.close()
            cnx.close()

# ========================================
# FUNCIONES DE ARTISTAS Y ÁLBUMES
# ========================================
def add_artist(nombre):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "INSERT INTO artistas (nombre) VALUES (%s)"
            cursor.execute(sql, (nombre,))
            cnx.commit()
            print(f"Artista '{nombre}' agregado exitosamente.")
        except Exception as e:
            print(f"Error al agregar el artista: {e}")
        finally:
            cursor.close()
            cnx.close()

def add_album(titulo, descripcion=None, anio_lanzamiento=None, portada_ruta=None):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "INSERT INTO albumes (titulo, descripcion, anio_lanzamiento, portada_ruta) VALUES (%s, %s, %s, %s)"
            cursor.execute(sql, (titulo, descripcion, anio_lanzamiento, portada_ruta))
            cnx.commit()

            id_creado = cursor.lastrowid
            print(f"Álbum '{titulo}' agregado con el ID: {id_creado}.")
            return id_creado
        except Exception as e:
            print(f"Error al agregar el álbum: {e}")
        finally:
            cursor.close()
```

```python
# ========================================
# FUNCIONES DE CANCIONES Y RELACIONES
# ========================================
def add_track(titulo, duracion_segundos=0, ruta_archivo=None):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "INSERT INTO canciones (titulo, duracion_segundos, ruta_archivo) VALUES (%s, %s, %s)"
            cursor.execute(sql, (titulo, duracion_segundos, ruta_archivo))
            cnx.commit()

            id_creado = cursor.lastrowid
            print(f"Canción '{titulo}' agregada con el ID: {id_creado}.")
            return id_creado
        except Exception as e:
            print(f"Error al agregar la canción: {e}")
        finally:
            cursor.close()
            cnx.close()

def link_track_to_album(id_album, id_cancion, numero_pista=None):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "INSERT INTO rel_tracks_album (id_album, id_cancion, numero_pista) VALUES (%s, %s, %s)"
            cursor.execute(sql, (id_album, id_cancion, numero_pista))
            cnx.commit()
            print(f"Canción ID {id_cancion} enlazada al álbum ID {id_album}.")
        except Exception as e:
            print(f"Error al vincular la canción al álbum: {e}")
        finally:
            cursor.close()
            cnx.close()

# ========================================
# FUNCIONES DE PLAYLISTS
# ========================================
def add_playlist(nombre, id_usuario, descripcion=None):
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()

            # 1. Crear la playlist
            sql_playlist = "INSERT INTO playlists (nombre, descripcion) VALUES (%s, %s)"
            cursor.execute(sql_playlist, (nombre, descripcion))
            id_playlist = cursor.lastrowid

            # 2. Vincularla al usuario como propietario
            sql_relacion = "INSERT INTO rel_user_playlist (id_usuario, id_playlist, es_propietario) VALUES (%s, %s, True)"
            cursor.execute(sql_relacion, (id_usuario, id_playlist))

            cnx.commit()
            print(f"Playlist '{nombre}' creada y vinculada al usuario ID {id_usuario}.")
        except Exception as e:
            cnx.rollback()
            print(f"Error al crear la playlist. Se deshicieron los cambios: {e}")
        finally:
            cursor.close()
            cnx.close()
```

```python
# ========================================
# FUNCIONES PARA MOSTRAR LOS DATOS
# ========================================
def show_albums():
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "SELECT id_album, titulo, descripcion, anio_lanzamiento FROM albumes"
            cursor.execute(sql)
            resultados = cursor.fetchall()

            if resultados:
                print("\n--- LISTA DE ÁLBUMES ---")
                for album in resultados:
                    id_album, titulo, descripcion, anio_lanzamiento = album
                    print("=" * 50)
                    print(f"ID: {id_album} | Título: {titulo} | Descripción: {descripcion or 'N/A'} | Año: {anio_lanzamiento or 'N/A'}")
                    print("=" * 50)
            else:
                print("No hay álbumes registrados.")
        except Exception as e:
            print(f"Error al mostrar los álbumes: {e}")
        finally:
            cursor.close()
            cnx.close()

def show_artists():
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "SELECT id_artista, nombre FROM artistas"
            cursor.execute(sql)
            resultados = cursor.fetchall()

            if resultados:
                print("\n--- LISTA DE ARTISTAS ---")
                for artista in resultados:
                    id_artista, nombre = artista
                    print(f"ID: {id_artista} | Nombre: {nombre}")
            else:
                print("No hay artistas registrados.")
        except Exception as e:
            print(f"Error al mostrar los artistas: {e}")
        finally:
            cursor.close()
            cnx.close()
```

```python
def show_albums():
    from conexion import connect_to_database

    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = """
                SELECT a.id_album, a.titulo, a.descripcion, a.anio_lanzamiento, c.titulo, r.numero_pista
                FROM albumes a
                LEFT JOIN rel_tracks_album r ON a.id_album = r.id_album
                LEFT JOIN canciones c ON r.id_cancion = c.id_cancion
                ORDER BY a.id_album, r.numero_pista
            """
            cursor.execute(sql)
            resultados = cursor.fetchall()

            if resultados:
                print("\n--- CATÁLOGO DE ÁLBUMES ---")

                album_actual = None

                for fila in resultados:
                    id_album, titulo_album, desc, anio, titulo_cancion, num_pista = fila

                    if id_album != album_actual:
                        print("\n" + "=" * 50)
                        print(f"💿 {titulo_album.upper()} (ID: {id_album})")
                        print(f"📅 Año: {anio or 'N/A'} | 📝 Desc: {desc or 'N/A'}")
                        print("-" * 50)
                        album_actual = id_album

                    if titulo_cancion:
                        pista = num_pista if num_pista else "-"
                        print(f"  🎵 {pista}. {titulo_cancion}")
                    else:
                        print("  (Este álbum aún no tiene canciones vinculadas)")

                print("=" * 50)
            else:
                print("No hay álbumes registrados en la base de datos.")

        except Exception as e:
            print(f"Error al mostrar los álbumes: {e}")
        finally:
            cursor.close()
            cnx.close()
def show_tracks():
    from conexion import connect_to_database
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "SELECT id_cancion, titulo, duracion_segundos FROM canciones"
            cursor.execute(sql)
            resultados = cursor.fetchall()

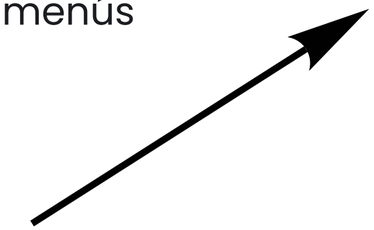
            if resultados:
                print("\n--- LISTA DE CANCIONES ---")
                for cancion in resultados:
                    id_cancion, titulo, duracion = cancion
                    minutos = duracion // 60
                    segundos = duracion % 60
                    print(f"ID: {id_cancion} | Título: {titulo} | Duración: {minutos}:{segundos:02d}")
            else:
                print("No hay canciones registradas.")
        except Exception as e:
            print(f"Error al mostrar las canciones: {e}")
        finally:
            cursor.close()
            cnx.close()
```

```python
def show_playlists():
    from conexion import connect_to_database
    cnx = connect_to_database()
    if cnx:
        try:
            cursor = cnx.cursor()
            sql = "SELECT id_playlist, nombre, descripcion FROM playlists"
            cursor.execute(sql)
            resultados = cursor.fetchall()

            if resultados:
                print("\n--- LISTA DE PLAYLISTS ---")
                for playlist in resultados:
                    id_playlist, nombre, desc = playlist
                    print(f"ID: {id_playlist} | Nombre: {nombre} | Descripción: {desc or 'N/A'}")
            else:
                print("No hay playlists registradas.")
        except Exception as e:
            print(f"Error al mostrar las playlists: {e}")
        finally:
            cursor.close()
            cnx.close()
```

ARCHIVO MAIN.PY

El archivo **main.py** es el punto de entrada de la aplicación y controla la interacción con el usuario a través de la consola. Se encarga de limpiar la pantalla en cada paso, pedir los datos por teclado y llamar a las funciones correspondientes mediante dos menús divididos por niveles de acceso:



1. Pantalla de Acceso (main)

Es una pantalla de bloqueo inicial que protege el sistema antes de mostrar el catálogo:

Opción 1 – Iniciar sesión: Solicita usuario y contraseña; si son correctos, da acceso al menú principal identificando el ID del usuario activo.

Opción 2 – Registrarse: Permite crear una cuenta nueva en la base de datos si el usuario ingresa por primera vez.

Opción 0 – Salir del programa: Finaliza la ejecución de la aplicación.

2. Menú Principal (menu_principal)

Una vez iniciada la sesión, despliega las opciones para administrar la biblioteca musical:

1. Agregar nuevo Artista:
Solicita el nombre del artista o banda y lo guarda en la base de datos.

2. Agregar nuevo Álbum:
Pide el título del disco y permite añadir opcionalmente su descripción y año de lanzamiento.

3. Agregar nueva Canción:
Registra una pista pidiendo su título, duración en segundos y la ruta del archivo.

4. Vincular Canción a un Álbum:
Solicita el ID del álbum, el ID de la canción y el número de pista para relacionarlos.

5. Crear nueva Playlist:
Pide un nombre y descripción para la lista y la vincula automáticamente al usuario que inició sesión.

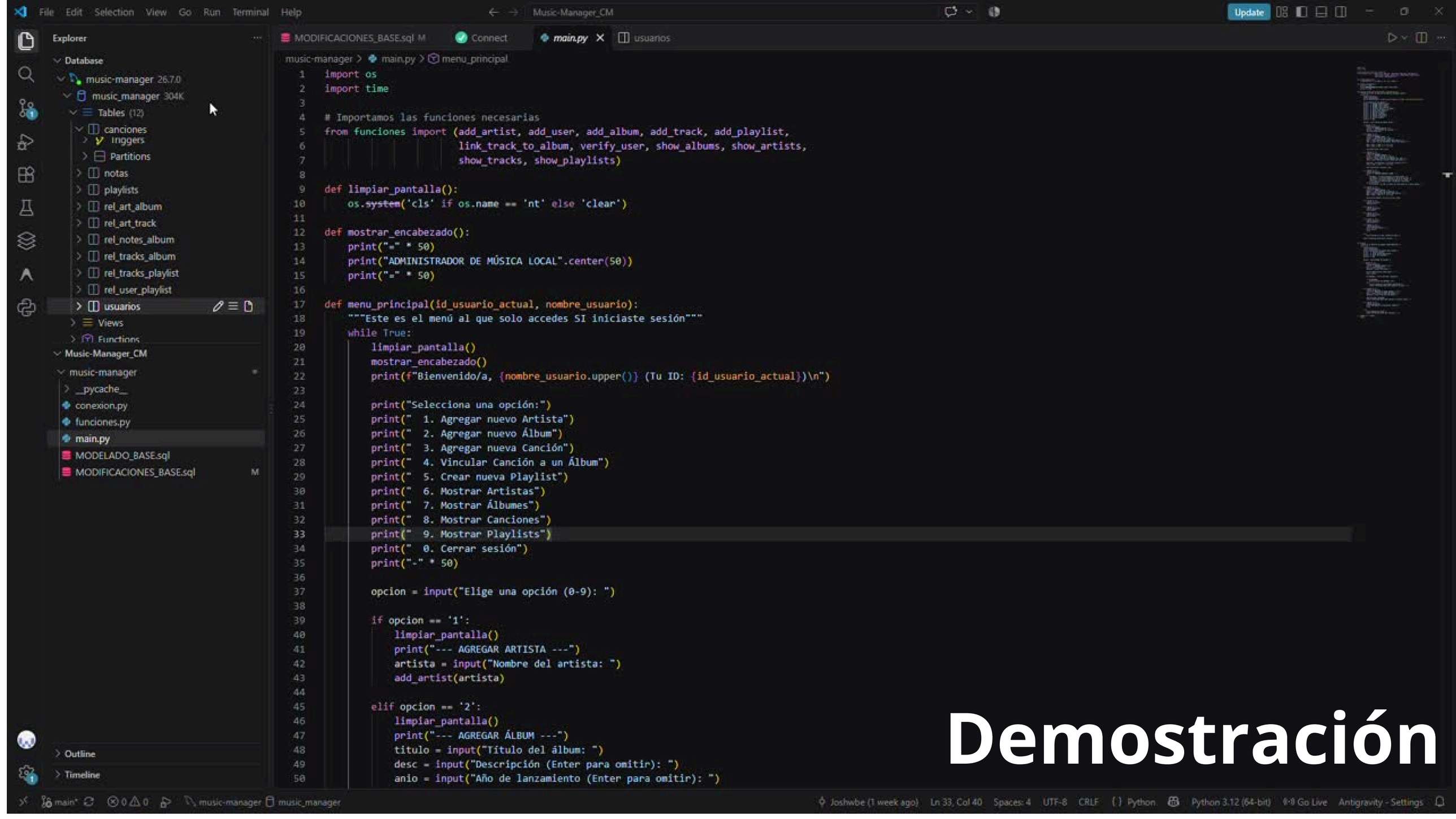
6. Mostrar Artistas:
Despliega en pantalla todos los artistas registrados.

7. Mostrar Álbumes:
Muestra cada álbum registrado junto con la lista de canciones que tiene asociadas.

8. Mostrar Canciones:
Imprime el catálogo completo de canciones con su duración en minutos y segundos.

9. Mostrar Playlists:
Lista todas las playlists guardadas en el sistema.

0. Cerrar sesión:
Regresa de forma segura a la pantalla inicial de bloqueo.


File Edit Selection View Go Run Terminal Help
Music-Manager_CM
Update
Explorer
Database
music-manager 26.7.0
music_manager 304K
Tables (12)
canciones
Triggers
Partitions
notas
playlists
rel_art_album
rel_art_track
rel_notes_album
rel_tracks_album
rel_tracks_playlist
rel_user_playlist
usuarios
Views
Functions
Music-Manager_CM
music-manager
__pycache__
conexion.py
funciones.py
main.py
MODELADO_BASE.sql
MODIFICACIONES_BASE.sql M
MODIFICACIONES_BASE.sql M
Connect
main.py
usuarios
music-manager > main.py > menu_principal
import os
import time
Importamos las funciones necesarias
from funciones import (add_artist, add_user, add_album, add_track, add_playlist,
link_track_to_album, verify_user, show_albums, show_artists,
show_tracks, show_playlists)
def limpiar_pantalla():
os.system('cls' if os.name == 'nt' else 'clear')
def mostrar_encabezado():
print("=" * 50)
print("ADMINISTRADOR DE MÚSICA LOCAL".center(50))
print("=" * 50)
def menu_principal(id_usuario_actual, nombre_usuario):
"""Este es el menú al que solo accedes SI iniciaste sesión"""
while True:
limpiar_pantalla()
mostrar_encabezado()
print(f"Bienvenido/a, {nombre_usuario.upper()} (Tu ID: {id_usuario_actual})\n")
print("Selecciona una opción:")
print(" 1. Agregar nuevo Artista")
print(" 2. Agregar nuevo Álbum")
print(" 3. Agregar nueva Canción")
print(" 4. Vincular Canción a un Álbum")
print(" 5. Crear nueva Playlist")
print(" 6. Mostrar Artistas")
print(" 7. Mostrar Álbumes")
print(" 8. Mostrar Canciones")
print(" 9. Mostrar Playlists")
print(" 0. Cerrar sesión")
print("-" * 50)
opcion = input("Elige una opción (0-9): ")
if opcion == '1':
limpiar_pantalla()
print("--- AGREGAR ARTISTA ---")
artista = input("Nombre del artista: ")
add_artist(artista)
elif opcion == '2':
limpiar_pantalla()
print("--- AGREGAR ÁLBUM ---")
titulo = input("Título del álbum: ")
desc = input("Descripción (Enter para omitir): ")
anio = input("Año de lanzamiento (Enter para omitir): ")
Outline
Timeline
main* 0 0 music-manager music_manager
Joshwbe (1 week ago) Ln 33, Col 40 Spaces: 4 UTF-8 CRLF Python Python 3.12 (64-bit) Go Live Antigravity - Settings
Demostración